Правительство Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

(НИУ ВШЭ)

ОТЧЕТ

ПО САМОСТОЯТЕЛЬНОЙ РАБОТЕ

В ФОРМАТЕ ЭССЕ С ИСПОЛЬЗОВАНИЕМ AI-АССИСТЕНТА

по дисциплине «Конструирование Баз Данных»

Студентка гр. БПИ245
Горбачева Маргарита
Валерьевна
«04» июня 2026 г.

Руководитель
_____ А.Д. Брейман
«____»_____2026г.

Москва, 2026

Содержание

ТЗ....3
Выполнение задания....8
Этап 1. Работа с основной моделью....8
Этап 2. Работа со второй моделью....10
Этап 3. Анализ производительности и эксплуатационных последствий....16
Этап 4. Техническое приложение. DDL, сценарии тестирования, SQL-запросы, пояснения к эксперименту....17
4.1 DDL-скрипты....17
4.1.1 Первичный ключ на UUID....17
4.1.2 Первичный ключ на BIGINT....18
4.2 Сценарий нагрузочного тестирования....18
4.3 Метрики для сравнения....18
4.4 Ожидаемые эффекты (теоретический прогноз)....19
4.5 Глубокий анализ: локальность вставки и деградация B-дерева (дополнительный параметр)....19
4.6 Выводы по дизайну эксперимента....20
Этап 5. Финальный текст доклада для выступления....21
Часть 1. Почему UUID так популярен?....21
Часть 2. Reality Check: Где аргументы в пользу UUID переоценены....21
Часть 3. Скрытые издержки UUID, которые часто недооценивают....22
Часть 4. Когда BIGINT (IDENTITY) — это прагматичный выбор по умолчанию....23
Часть 5. Когда UUID действительно оправдан (и как его использовать правильно)....23
Часть 6. Итоговая инженерная рекомендация....24
Заключение....24

«Почему BIGSERIAL часто оказывается более прагматичным выбором, чем UUID, даже в современных системах»

ТЗ

Задача — подготовить этот доклад с помощью AI-ассистентов, но не как готовый текст “из нейросети”, а как результат управляемого инженерного исследования с проверкой и перепроверкой выводов.

Цель задания

Задание проверяет не только знание темы первичных ключей в PostgreSQL, но и вашу способность:

* формулировать исследовательскую задачу;
* использовать AI-ассистента как инструмент анализа, а не как “генератор ответа”;
* находить сильные аргументы противоположной позиции;
* переводить рассуждение в эксперимент;
* проверять выводы одной модели другой моделью;
* отделять убедительный текст от инженерно проверяемого вывода;
* формулировать итоговую позицию с границами применимости.

Что нужно сдать:

В ходе данной работы необходимо приготовить единый документ, включающий:

1. Полный лог работы с основной AI-моделью

Все ключевые запросы и ответы, на которых строилось исследование.

2. Лог независимой верификации второй моделью

Отдельный блок диалога, где промежуточные выводы были проверены другой LLM.

3. Краткий комментарий к обоим логам

На 8–15 предложений:

* что мы делали на каждом этапе;

* какие ответы оказались полезными;
* где AI ошибался, упрощал или выдавал спорные выводы;
* как мы корректировали направление работы;
* что именно изменилось после независимой верификации.

4. Техническое приложение
DDL, сценарии тестирования, SQL-запросы, пояснения к эксперименту.

5. Финальный текст доклада
Цельный, структурированный текст выступления.

Обязательная структура работы:

Этап 1. Сбор сильных аргументов в пользу UUID

Сформулировать запрос к основной AI-модели так, чтобы она собрала наиболее сильные аргументы в пользу UUID как первичного ключа.

Важно: на этом этапе нельзя сразу просить “доказать, что UUID плох”. Наша задача — сначала получить сильную версию противоположной позиции.

В логе должно быть видно:

* какие аргументы предложил AI;
* какой из них мы считаем самым сильным;
* почему именно он кажется нам наиболее серьезным.

Этап 2. Контраргументы и поиск альтернатив

Возьмем самый сильный аргумент из предыдущего этапа и попросим AI-ассистента:
* найти контраргументы;
* указать ограничения этого аргумента;

* предложить альтернативные решения, которые позволяют решить ту же проблему без обязательного использования UUID.

Например, если выбран аргумент «глобальная уникальность в распределенной системе»,
нужно исследовать, какие еще подходы могут использоваться:
* последовательности в одном контуре;
* составные ключи;
* внешние публичные идентификаторы;
* UUID только на внешнем уровне;
* упорядоченные идентификаторы;
* и т.д.

На этом этапе важно показать, что мы умеем не просто спорить, а декомпозировать тезис и искать инженерные альтернативы.

Этап 3. Анализ производительности и эксплуатационных последствий

Попросим AI-ассистента подготовить два варианта схемы для PostgreSQL:
* таблица `event_log` с первичным ключом на UUID;
* таблица `event_log` с первичным ключом на BIGINT / BIGSERIAL.

После этого попросим AI:
* предложить сценарий нагрузочного тестирования;
* объяснить, какие метрики имеет смысл сравнивать;
* указать, какие эффекты можно ожидать на уровне хранения и индексации.

В технической части работы обязательно должно быть:
* два DDL-скрипта;
* сценарий массовой вставки данных;
* описание того, что именно сравнивается;
* хотя бы один дополнительный параметр, кроме скорости вставки.

Минимум нужно сравнить:
* массовую вставку (`INSERT`).

Дополнительно выбрать хотя бы один пункт:
* размер индекса;
* размер таблицы;
* влияние на вторичные индексы;
* локальность вставки и влияние на структуру индекса;
* читаемость и удобство эксплуатации;
* сравнение случайных UUID и упорядоченных идентификаторов;
* влияние на отладку, журналирование и анализ данных.

Важно: если мы не запускаем реальный тест, а ограничиваемся проектированием сценария, это должно быть честно указано. В таком случае оценивается качество постановки эксперимента, а не “выдуманные результаты”.

Этап 4. Независимая верификация второй моделью

Перед написанием финального текста передаем наши промежуточные выводы другой LLM и попросим ее выступить в роли критически настроенного технического рецензента.

Ее задача — не “помочь красиво оформить ответ”, а попытаться разрушить вашу позицию и найти:
* технические ошибки;
* логические натяжки;
* неявные допущения;
* слабые места в контраргументах;
* проблемы в DDL или нагрузочном сценарии;
* сценарии, в которых ваши выводы перестают работать.

В логе верификации должно быть видно:
* какой именно текст, тезисы или артефакты вы дали второй модели;
* как вы сформулировали запрос на критику;
* какие замечания она сделала.

После этого мы обязаны отдельно указать:
1. Какие 2–3 замечания второй модели были наиболее ценными
2. С какими замечаниями вы согласились, а с какими нет
3. Что именно вы изменили в итоговой позиции после этой проверки

Важно: вторая модель не должна быть “еще одним генератором готового ответа”. Ее функция — независимая верификация и разрушение слишком гладкого вывода.

Финальный текст доклада

На основе диалога с основной моделью, независимой верификации и собственного анализа подготовим текст выступления для технического митапа.

В докладе мы должны:
1. Показать, какие аргументы в пользу UUID обычно считаются самыми сильными.
2. Разобрать, почему часть этих аргументов может быть переоценена.
3. Показать, какие издержки UUID часто недооцениваются:
   * производительность;
   * размер индексов;
   * сложность эксплуатации;
   * ухудшение читаемости;
   * архитектурная избыточность в части сценариев.
4. Сформулировать, в каких случаях BIGSERIAL оказывается более прагматичным выбором.
5. Отдельно указать, в каких сценариях UUID все же оправдан.
6. Учесть замечания второй модели и показать, как независимая верификация повлияла на наш итоговый вывод.
7. Сделать итоговый вывод не в форме лозунга, а в форме инженерной рекомендации с границами применимости.

Важное ограничение

Наша задача — не “доказать любой ценой, что UUID плох”, а провести честный инженерный разбор.

Итоговая позиция может быть жесткой, но она должна быть:
* аргументированной;
* проверяемой;
* технически осмысленной;
* не сводящейся к вкусовщине.

Что особенно важно в этом задании
Будет оцениваться не только финальный текст, но и качество процесса решения:

* как мы формулировали запросы к основной модели;
* умеем ли мы вытаскивать сильный аргумент противоположной стороны;
* умеем ли мы заставить AI искать слабые места в собственных ответах;
* переходим ли мы от общих слов к инженерному эксперименту;
* умеем ли мы замечать спорные или поверхностные ответы;
* умеем ли мы использовать вторую модель как инструмент критической проверки, а не как “еще один источник красивого текста”;
* умеем ли мы пересобрать собственный вывод после критики.

Выполнение задания

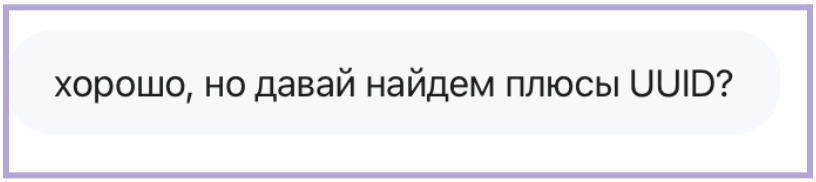
Этап 1. Работа с основной моделью

Итак, мы начали с того, что первой моделью я решила выбрать open-source ИИ - Qwen. Я отрпавила ему запрос, который состоит из общей фразы - «Почему BIGSERIAL часто оказывается более прагматичным выбором, чем UUID, даже в современных системах». Нейросеть ответила распространенно - вначале был выдвинут общий тезис -

Выбор между `BIGSERIAL` (8-байтное целое число с автоинкрементом) и `UUID` (16-байтный универсальный уникальный идентификатор) — это классическая дилемма при проектировании баз данных. Несмотря на популярность UUID в современных распределенных системах, `BIGSERIAL` (и его современный аналог `GENERATED ALWAYS AS IDENTITY`) часто остается более прагматичным выбором по нескольким фундаментальным причинам.

Становится очевидно, что Qwen будет “топить” за BIGSERIAL. Далее он приводит плюсы и минусы обоих подходов. Хорошо, давайте попробуем оспорить его мнение, попытаемся выудить информацию, где же все же плюсы.

Итак, делаем запрос:



Теперь ИИ-моделька выдает нам, что, действительно, плюсы у UUID есть, несмотря на то, что BIGSERIAL — это король производительности и простоты в монолитных системах, то UUID — это абсолютный король распределенных архитектур, безопасности и гибкости. Qwen выявил главные плюсы и положительные особенности использования UUID.

1)

1. Глобальная уникальность без централизации

Это самое главное архитектурное преимущество. Чтобы сгенерировать `BIGSERIAL`, приложению **обязательно** нужно обратиться к базе данных (или к сервису, который хранит последовательность).

- **С UUID:** Любой микросервис, любой сервер, любой клиент может сгенерировать уникальный идентификатор *локально*, не спрашивая разрешения у центрального узла.
- **Где критично:** В микросервисной архитектуре, при шардировании баз данных, в системах с несколькими активными-master нодами (Multi-Master), где записи создаются параллельно в разных дата-центрах.

2)

2. Сокрытие бизнес-метрик и конкурентная разведка

Последовательные ID (1, 2, 3...) — это дыра в аналитической безопасности.

- Если конкурент зарегистрируется в вашем сервисе, а через неделю сделает это снова, он легко узнает, сколько у вас появилось новых пользователей или заказов за неделю (разница между ID).
- Злоумышленник может понять, насколько популярен ваш новый продукт, просто посмотрев на ID последней созданной записи.
- **С UUID:** ID выглядят как случайный шум. Невозможно понять, сколько записей в таблице и как быстро они создаются.

3)

3. Защита от перебора (смягчение IDOR)

Хотя UUID **не заменяет** собой нормальную проверку прав доступа (ACL), они сильно усложняют жизнь хакерам при атаках типа **IDOR** (Insecure Direct Object Reference).

- Если в URL документа лежит `/docs/154`, хакер может просто написать скрипт, который переберет все ID от 1 до 10000 и скачает все чужие документы, если разработчик забыл проверить права.
- С UUID (`/docs/a0eebc99-9c0b...`) пространство для перебора настолько огромно (2^122 для случайных UUID), что брутфорс становится бессмысленным.

4)

4. Бесшовное слияние и миграция данных

Представьте, что ваша компания купила другую компанию, и вам нужно слить их базу данных с вашей. Или вам нужно перенести данные из staging-окружения в production.

- **С BIGSERIAL:** Это кошмар DBA. При слиянии таблиц возникнут конфликты первичных ключей. Вам придется генерировать новые ID для одной из баз и **обновлять все внешние ключи (Foreign Keys)** во всех связанных таблицах.
- **С UUID:** Вы можете просто скопировать строки из одной базы в другую. Конфликтов не будет, связи не нарушатся.

Таким образом, можно проанализировать ответ ИИ и, согласно ТЗ, выбрать самый сильный аргумент. Мне кажется, что первый аргумент выигрывает у всех остальных, т.к. по своим свойствам он важнее, чем сокрытие бизнес-метрик, смягчения перебора и тп.

Основными важными характеристиками первого поинта я считаю:

1. Децентрализованная генерация UUID - не просто плюс, а фундамент, на котором держится вся асинхронная и распределенная архитектура.

2. Когда мы используем UUID, идентификатор становится свойством самой бизнес-сущности. Мы можем создать объект в памяти, присвоить ему UUID, передать его по шине сообщений, сохранить в локальный кэш, отправить в очередь - и он уже имеет свой уникальный “паспорт”.

Этап 2. Работа со второй моделью

Теперь в качестве второй модели берем нашу любимую Алису AI. Как и в первом этапе, спрашиваем у Алисы, «Почему BIGSERIAL часто оказывается более прагматичным выбором, чем UUID, даже в современных системах». Ответ Алисы:

> Действительно, во многих сценариях BIGSERIAL (или `GENERATED ALWAYS AS IDENTITY` в современных PostgreSQL) оказывается прагматичнее UUID. Я бы сказала так: дело не в том, что UUID плох — просто он решает специфические задачи, а в большинстве типовых проектов эти задачи просто не возникают. Я подобрала главные аргументы, почему BIGSERIAL часто выигрывает. queryplane.com +2

Точно так же, как и Qwen, Алиса предлагает посмотреть на ряд преимуществ BIGSERIAL перед UUID. Далее мы закинули Алисе ответы Qwen’а и попросили проанализировать их. Алиса предложила схожие ответы со сносками на ответы Qwen’а.

1)

> **Основные преимущества UUID**
>
> **1. Глобальная уникальность**
>
> Главное свойство UUID: вероятность коллизии (совпадения двух идентификаторов) пренебрежимо мала. UUID уникален не в рамках одной таблицы или базы данных, а *глобально* — во времени и пространстве. Это критически важно при:
>
> - объединении данных из разных баз;
> - работе с распределёнными системами;
> - синхронизации офлайн-данных.

2)

2. Децентрализованная генерация

UUID можно генерировать в любом месте без координации с центральным сервером:

- каждый микросервис в архитектуре может создавать свои UUID;
- мобильные приложения генерируют UUID для записей в офлайн-режиме, а затем синхронизируют их с сервером;
- разные узлы кластера БД создают уникальные ID независимо друг от друга.

3)

3. Безопасность и конфиденциальность

Последовательные числовые ID (как BIGSERIAL) предсказуемы — злоумышленник может:

- перебирать ID в URL (`site.com/user/123` , `site.com/user/124` и т.д.);
- оценить объём данных по диапазону ID;
- провести enumeration-атаку.

UUID (особенно версии 4) непредсказуемы и усложняют такие атаки.

Как мне кажется, пункты схожи с ответами Qwen’a. Однако, 5 пункт очень отличается от ответов прошлой нейронки. По моему мнению, данный пункт достаточно бесполезный и странный. Алиса не нашла недостатков Qwen’a, написав, что она немного бы переформулировала некоторые тезисы.

Итак, согласно ТЗ перечисляем контраргументы самого мощного аргумента UUID от первой нейронки, который, напомню: глобальная уникальность без централизации. Алисина рецензия выдает:

Контраргументы

- **Риск коллизий (хоть и малый).** Технически UUID не гарантируют уникальность «по построению» — они лишь делают вероятность коллизии пренебрежимо малой. В подавляющем большинстве случаев этим можно пренебречь, но в системах экстремаального масштаба (например, при генерации миллиардов ID в секунду на протяжении лет) или при использовании некачественных генераторов (слабый PRNG, усечение битов для «коротких» ID) риск становится ощутимым. xerobit.dev +2
- **Производительность и накладные расходы.** Генерация UUID (особенно версий с криптографической случайностью) — операция чуть более ресурсоёмкая, чем инкремент числа. В высоконагруженных системах, где ID генерируются тысячами раз в секунду, это накапливается. Кроме того, если хранить UUID как строку (а не в бинарном виде), расход памяти и нагрузка на индексы растут значительно. dev.to +1
- **Проблемы с индексами в БД.** Если использовать в качестве первичного ключа случайный UUID (например, v4), вставки будут распределяться по всему индексу. Это провоцирует фрагментацию B-дерева, дополнительные page splits и снижает общую пропускную способность при интенсивной записи. supabase.com +2
- **Сложность отладки и работы с данными.** Длинный, нечитаемый ID сложнее анализировать в логах, обсуждать с коллегами или использовать в интерфейсах по сравнению с числовым.

Эти контраргументы особенно сильны при:

- В системах с очень высокой частотой генерации ID, где каждый цикл генерации на счету.
- В базах данных с интенсивными операциями записи, где фрагментация индексов напрямую бьёт по производительности.
- В приложениях, где читаемость ID критична (например, в пользовательских URL, логах, инструментах отладки).
- Когда архитектура системы на самом деле не требует глобальной уникальности — достаточно локальной.

Ну и, собственно, альтернативные решения, которые позволяют решить ту же проблему без обязательного использования UUID:

- **Snowflake (и производные, вроде Sonyflake).** Это 64-битный идентификатор, который комбинирует временную метку, ID узла (worker ID) и счётчик (sequence number). Он даёт локальную уникальность без централизации, при этом остаётся компактным (8 байт, как BIGSERIAL), сортируемым по времени и дружелюбным для индексов. Требует синхронизации часов между узлами и распределения worker ID, но это управляемые накладные расходы. T tutorialpedia.org +1
- **ULID (Universally Unique Lexicographically Sortable Identifier).** По сути, это тоже комбинация временной метки и случайного компонента, но представленная в виде строки в кодировке base32. Он лексикографически сортируем (что удобно для запросов «последние N записей»), при этом может генерироваться независимо на разных узлах. T tutorialpedia.org +1
- **UUIDv7.** Это эволюция идеи: в наиболее значимые биты UUID заложена временная метка, что делает последовательность монотонной. В результате вставки в индекс идут последовательно — фрагментация сводится к минимуму, и по производительности такой подход приближается к BIGSERIAL, сохраняя при этом глобальную уникальность. DEV dev.to +1
- **Гибридный подход.** Иногда оптимально использовать два ID: компактный числовой (BIGSERIAL) как внутренний ключ для JOIN и производительности, и отдельный публичный UUID (или другой децентрализованный ID) как внешний идентификатор для API, логов или обмена между системами. sqllab.ru

Алиса резко отреагировала на следующие ответы Qwen’a:

1. Технические ошибки

Ошибка А: Механизм вторичных индексов в PostgreSQL

- **Что было сказано ранее:** «Вторичные индексы хранят значение первичного ключа... 16 байт против 8 байт на каждую запись во всех вторичных индексах».
- **Реальность:** Это поведение характерно для **MySQL (InnoDB)**, где вторичные индексы действительно хранят значение первичного ключа. В **PostgreSQL** (который использует кучу/heap-таблицы) вторичный индекс хранит индексируемые колонки + `ctid` (Tuple ID), который занимает ровно **6 байт** (номер блока + смещение в блоке), *независимо* от размера первичного ключа.
- **Исключение:** `ctid` заменяется на значение PK только если индекс объявлен как `UNIQUE`, или если явно используется синтаксис `INCLUDE (id)`. Таким образом, аргумент о том, что *каждый* вторичный индекс раздувается на 8 байт из-за UUID, для стандартных неуникальных индексов в PostgreSQL **неверен**.

Ошибка Б: Избыточность в DDL

- **Что было сказано:** `CREATE EXTENSION IF NOT EXISTS pgcrypto;` перед использованием `gen_random_uuid()`.
- **Реальность:** Начиная с PostgreSQL 13, функция `gen_random_uuid()` является встроенной (core function) и не требует подключения расширения `pgcrypto`. Это устаревшая практика, которая может сбить с толку при чтении кода.

---

2. Логические натяжки и неявные допущения

Натяжка А: Математика «раздувания» таблицы на 5–10%

- **Допущение:** Я утверждал, что таблица с UUID будет больше на 5–10%.
- **Реальность:** Это справедливо *только* для очень «узких» таблиц (например, только `id` и `status`). В моем же сценарии есть колонка `payload JSONB`. Даже минимальный JSON-объект весит десятки байт, а с учетом заголовков кортежа PostgreSQL (23 байта), выравнивания (padding) и возможного TOAST-сжатия, разница между 8 и 16 байтами на ключ составляет **менее 1–2%** от общего размера строки. Масштабировать эту разницу до 10% без учета реального размера строки — логическая ошибка.

3. Слабые места в контраргументах

Слабость А: «Защита от перебора (IDOR)»

- **Аргумент:** UUID усложняет жизнь хакерам при переборе ID.
- **Контраргумент:** Это классическая «безопасность через неясность» (Security by Obscurity). Если в приложении есть уязвимость IDOR (отсутствие проверки прав: `WHERE user_id = current_user_id`), то утечка UUID через Referer-заголовок, логи фронтенда, GraphQL-ответы или ссылки в письмах делает эту «защиту» бесполезной. Профессиональный аудит безопасности не считает UUID защитой от IDOR.

Слабость Б: «Удобство слияния баз данных»

- **Аргумент:** С UUID можно просто скопировать строки, и конфликтов не будет.
- **Контраргумент:** В реальных enterprise-системах слияние баз данных (data migration/merge) почти никогда не бывает простым `COPY`. Всегда требуется трансформация данных, разрешение бизнес-конфликтов (например, два пользователя с разными email, но одинаковыми UUID из разных систем), обновление внешних ключей и маппинг. Простота копирования строк — это иллюзия, которая работает только в академических примерах.

Итоговый ответ Алисы по критике:

> Предыдущий ответ давал хорошую *общую* картину, но грешил переносом архитектурных паттернов из MySQL в PostgreSQL (ошибка с индексами), преувеличением влияния размера ключа на общий размер строки и предложением слишком «тепличного» сценария тестирования, который не выявляет реальные проблемы конкуренции последовательностей. Настоящий senior-подход требует учета `ctid`, `FILLFACTOR`, поведения NVMe и рисков синхронизации времени для UUIDv7.

Этап 3. Анализ производительности и эксплуатационных последствий

Согласно ТЗ, нам необходимо дать краткий комментарий к обоим логам на 8–15 предложений:

* что мы делали на каждом этапе;
* какие ответы оказались полезными;
* где AI ошибался, упрощал или выдавал спорные выводы;
* как мы корректировали направление работы;
* что именно изменилось после независимой верификации.

1) На каждом из предыдущих этапов мы просили модель ответить на главный вопрос данного эссе - «Почему BIGSERIAL часто оказывается более прагматичным выбором, чем UUID, даже в современных системах». После ответа модельки, который склонялся в сторону использования BIGSERIAL, мы просили выдать нам плюсы UUID. Далее мы проверяли ответы другой нейросети на корректность ответов первой. Т.е. вторая нейросеть выступала неким ревизором для первой.
2) В целом, ответы Qwen’а мне показались более приспособленными к жизни. Однако и ответы Алисы были не промах. Полезными ответы были про глобальную уникальность, про децентрализованную генерацию, это все действительно так, как говорилось на наших лекциях и семинарских занятиях.
3) Мне показался вывод Алисы - совместимость между системами - странным ответом, тк это, по большей части, не относится к характеристикам, влияющим на корректную работу UUID.

4) Корректировали работу и ответы ИИ мы с помощью слов и корректных формулировок 🙂 Выдавали Алисе ответы Qwen'а, чтобы она, в свою очередь, выдавала "свое мнение по ответам Qwen'а с пояснениями".

Этап 4. Техническое приложение. DDL, сценарии тестирования, SQL-запросы, пояснения к эксперименту.

4.1 DDL-скрипты

Для чистоты эксперимента создадим таблицу event_log, которая типична для систем телеметрии или аудита. Мы добавим вторичный индекс по event_type, чтобы оценить влияние размера первичного ключа на него.

4.1.1 Первичный ключ на UUID

```sql
-- Включаем расширение для генерации UUID (встроено в PG 13+)
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE event_log_uuid (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type VARCHAR(50) NOT NULL,
    payload JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Вторичный индекс для частых запросов по типу события и времени
CREATE INDEX idx_event_log_uuid_type_time
ON event_log_uuid (event_type, created_at);
```

4.1.2 Первичный ключ на BIGINT

```sql
CREATE TABLE event_log_bigint (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_type VARCHAR(50) NOT NULL,
    payload JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Аналогичный вторичный индекс
CREATE INDEX idx_event_log_bigint_type_time
ON event_log_bigint (event_type, created_at);
```

4.2 Сценарий нагрузочного тестирования

Для оценки используем скрипт на Python (с библиотекой asyncpg или psycopg2 и методом execute_values для пакетной вставки), так как это наиболее точно имитирует реальное поведение backend-приложения.

```
SELECT count(*) FROM event_log WHERE event_type = 'login' AND created_at > NOW() - INTERVAL '1 day'
```

- для оценки эффективности индексов

4.3 Метрики для сравнения

1. Throughput массовой вставки (rows/sec): Сколько строк в секунду способна принять таблица.
2. Физический размер таблицы (pg_table_size): Место, занимаемое самими данными (heap).
3. Размер первичного индекса (pg_indexes_size для PK): Место, занимаемое B-деревом первичного ключа.
4. Размер вторичного индекса: Критически важная метрика, так как в PostgreSQL вторичные индексы хранят значение первичного ключа в качестве указателя на строку (в случае уникальных индексов или при использовании INCLUDE, либо как скрытый указатель).

4.4 Ожидаемые эффекты (теоретический прогноз)

Основываясь на архитектуре PostgreSQL, мы ожидаем следующие результаты:

1. Скорость массовой вставки (INSERT)

- BIGINT: Ожидается высокая и стабильная скорость. Вставка происходит монотонно, новые страницы добавляются в конец индекса и файла данных. Механизм IDENTITY с кэшированием (по умолчанию cache=1, но его можно увеличить) создает минимальную конкуренцию.
- UUID (v4): Ожидается снижение скорости на 15–30% при высоких нагрузках. Случайные значения заставляют PostgreSQL постоянно искать нужную страницу в B-дереве, загружать её с диска в RAM, выполнять вставку и, что самое важное, инициировать разделение страниц (page splits).

2. Влияние на хранение и индексы

- Размер таблицы: Таблица с UUID будет больше примерно на 5-10% (8 байт разницы на строку + возможное выравнивание padding в структуре строки PostgreSQL).
- Размер индексов: Это главный "скрытый налог". Индекс по BIGINT будет значительно компактнее. Более того, вторичный индекс idx_event_log_bigint_type_time будет меньше, чем его аналог для UUID, потому что PostgreSQL хранит значение первичного ключа внутри вторичного индекса для разрешения дубликатов и переходов к heap-строке. 16 байт против 8 байт на каждую запись во *всех вторичных* индексах.

4.5 Глубокий анализ: локальность вставки и деградация B-дерева (дополнительный параметр)

Проблема случайных UUID (v4): B-дерево PostgreSQL оптимизировано для последовательных данных. Когда вы вставляете случайный UUID, он с равной вероятностью может попасть в любую часть дерева.

1. Page Splits (Разделение страниц): Если страница индекса (обычно 8 КБ) заполнена, вставка нового ключа в её середину заставляет PostgreSQL разделить её на две страницы по 50%. Это удваивает логический размер данных в индексе, создавая "пустоты" (fragmentation).
2. Падение эффективности кэша (Cache Churn): Из-за фрагментации и случайного доступа, полезные страницы индекса постоянно вытесняются из shared_buffers (оперативной памяти) бесполезными страницами, которые больше никогда не понадобятся. Это приводит к резкому росту дискового I/O (random reads/writes) при росте таблицы.

Прагматичный компромисс 2026 года: UUID v7 Если мы модифицируем Вариант А, используя uuid_generate_v7() (или аналогичную функцию, генерирующую UUID на основе Unix-времени в старших битах), мы получаем "лучшее из двух миров":

- Сохраняется децентрализация: ID можно генерировать на стороне приложения без похода в БД.
- Восстанавливается локальность: Поскольку старшие биты содержат время, новые UUID всегда *больше* предыдущих. Вставка происходит строго в конец B-дерева, точно так же, как при использовании BIGINT. Page splits практически исчезают, а размер индекса остается предсказуемым и компактным.

4.6 Выводы по дизайну эксперимента

Данный сценарий тестирования позволяет не просто получить абстрактные цифры "секунд", а наглядно продемонстрировать фундаментальные компромиссы:

1. Если бизнесу критична децентрализация генерации ID (микросервисы, offline-клиенты), выбор должен пасть на UUID.
2. Однако, чтобы избежать катастрофической деградации производительности, описанной в Части 5, категорически не рекомендуется использовать случайные UUID (v4) для первичных ключей в высоконагруженных таблицах.

3. Рекомендация: Использовать BIGINT GENERATED ALWAYS AS IDENTITY для монолитов, или UUID v7(или ULID) для распределенных систем, чтобы сохранить преимущества децентрализации, не жертвуя локальностью записи и размером индексов.

Этап 5. Финальный текст доклада для выступления

Часть 1. Почему UUID так популярен?

а)Абсолютная децентрализация генерации. Это «киллер-фича». UUID можно сгенерировать на любом микросервисе, в очереди сообщений или даже в мобильном приложении пользователя *до*отправки запроса в БД. Нам не нужен центральный координатор, не нужна сеть, не нужна блокировка. Сущность получает свой уникальный «паспорт» в момент рождения в доменной модели.

б) Бесшовное слияние данных. При миграции, шардировании или объединении баз данных (например, при M&A) конфликтов первичных ключей не возникает. Мы можем делать COPY таблиц без перестройки всех внешних ключей (Foreign Keys).

в)Смягчение (но не решение!) проблемы IDOR. Случайный UUID делает бессмысленным тупой перебор (/api/users/1, /api/users/2), так как пространство поиска составляет 2^122.

Часть 2. Reality Check: Где аргументы в пользу UUID переоценены.

здесь мы должны быть честны и признать, что некоторые популярные аргументы не выдерживают критики при глубоком техническом анализе PostgreSQL. Независимая верификация выявила следующие логические натяжки:

- Миф о раздувании *всех* вторичных индексов. Часто говорят: «UUID занимает 16 байт, поэтому *каждый* вторичный индекс становится больше на 8 байт». Это неверно для PostgreSQL. В отличие от MySQL (InnoDB), PostgreSQL использует кучу (heap). Вторичный индекс хранит индексируемые колонки и системный указатель ctid (Tuple ID),

который занимает ровно 6 байтнезависимо от размера первичного ключа. PK попадает во вторичный индекс *только* если индекс объявлен как UNIQUE или используется синтаксис INCLUDE (id).

- Миф о 10% раздувании таблицы. Если у вас узкая таблица из двух колонок, разница 8 vs 16 байт существенна. Но в реальной таблице с VARCHAR, TIMESTAMP и JSONB (который имеет свои заголовки и возможный TOAST), разница в 8 байт составляет менее 1–2% от размера строки. Масштабировать эту цифру до двузначных процентов — математическая ошибка.
- Миф о безопасности. UUID — это «безопасность через неясность» (Security by Obscurity). Если у вас нет проверки прав доступа (ACL: WHERE owner_id = current_user), UUID не спасет. Он легко утекает через Referer-заголовки, логи фронтенда, GraphQL-ответы или ссылки в email-рассылках.

Часть 3. Скрытые издержки UUID, которые часто недооценивают

Даже с учетом поправок из части 2, UUID несет в себе реальные инженерные издержки:

1. Деградация B-дерева и Cache Churn (для UUID v4). Случайные UUID заставляют PostgreSQL вставлять новые ключи в случайные места индекса. Это вызывает частые разделения страниц (page splits), фрагментацию и, что хуже всего, вытеснение полезных страниц из shared_buffers(оперативной памяти). Растет случайный дисковый I/O.
2. Операционная сложность и читаемость. Попробуйте найти конкретную запись в логах приложения или вручную выполнить SELECT в консоли, диктуя a0eebc99-9c0b-4ef8.... Это прямой путь к человеческим ошибкам при отладке (troubleshooting).
3. Архитектурная избыточность. Использование UUID для первичного ключа в простом монолите, который никогда не будет шардирован, — это over engineering. Вы платите сложностью за возможность, которой никогда не воспользуетесь.
4. Проблема тестирования. Нагрузочное тестирование UUID часто проводят некорректно: пакетная вставка (INSERT по 10 000 строк) скрывает реальную проблему. Реальная боль BIGINT — это

конкуренция (contention) при *одиночных* высококонкурентных вставках, а UUID страдает от фрагментации при *любом* паттерне записи.

Часть 4. Когда BIGINT (IDENTITY) — это прагматичный выбор по умолчанию

BIGINT GENERATED ALWAYS AS IDENTITY (именно он, а не устаревший BIGSERIAL) должен быть выбором по умолчанию, если:

- У вас монолитная архитектура или классическая схема Master-Replica без мульти-мастер записи.
- Таблица является heavily write-optimized (например, логирование, телеметрия), где важна максимальная скорость последовательной записи (append-only).
- Вам критична читаемость данных в БД для быстрой отладки и поддержки.
- Вы не планируете объединять эту таблицу с данными из совершенно независимых внешних систем.

Часть 5. Когда UUID действительно оправдан (и как его использовать правильно)

UUID необходим, когда бизнес-требования ломают возможности централизованной генерации:

1. Распределенные системы и микросервисы, где несколько узлов должны создавать записи независимо и параллельно.
2. Offline-first и мобильные приложения, где клиент должен создать сущность и иметь ее стабильный ID до момента появления сети и синхронизации с сервером.
3. Событийные шины (Event Sourcing / Kafka), где сообщение должно иметь глобальный идентификатор еще до попадания в базу данных.

Вставка происходит в конец B-дерева, что сводит на нет проблему page splits и сохраняет локальность данных, приближаясь по производительности к BIGINT.

Часть 6. Итоговая инженерная рекомендация

Вместо лозунгов, предлагаю использовать следующий чеклист при проектировании схемы БД:

1. Default Rule: Начинайте с BIGINT GENERATED ALWAYS AS IDENTITY. Это самый дешевый, быстрый и предсказуемый вариант для 80% задач.
2. Distributed Rule: Если архитектура *требует* децентрализованной генерации ID (клиент, микросервис, очередь), используйте UUID v7 (или ULID). Откажитесь от UUID v4 для первичных ключей в таблицах с высокой интенсивностью записи.
3. Security Rule: Никогда не полагайтесь на UUID как на механизм защиты от несанкционированного доступа. Реализуйте корректные проверки прав (ACL) на уровне приложения или базы данных (например, через RLS в PostgreSQL).
4. Index Rule: Если вы вынуждены использовать UUID v4, обязательно создавайте первичный индекс с параметром FILLFACTOR = 80 или 90, чтобы оставить резерв места на страницах и смягчить эффект фрагментации.

Заключение

Хорошая инженерия — это не использование самых модных технологий, а осознанный выбор инструмента, чьи компромиссы наилучшим образом соответствуют конкретным ограничениям вашей системы.